



云计算

CLOUD COMPUTING Third Edition

第2章

Google云计算原理与应用（二）

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

2.3 分布式锁服务Chubby

• 初步了解Chubby

Chubby是Google设计的提供粗粒度锁服务的一个文件系统，它基于松耦合分布式系统，解决了分布的一致性问题。



通过使用Chubby的**锁服务**，用户可以确保数据操作过程中的一致性



Chubby作为一个稳定的**存储系统**存储包括元数据在内的小数据



Google内部还使用Chubby进行**名字服务** (Name Server)

2.3 分布式锁服务Chubby

- 初步了解Chubby

本节首先简要介绍Paxos算法，因为Chubby内部一致性问题的实现用到了Paxos算法；

然后围绕Chubby系统的设计和实现展开讲解。

2.3 分布式锁服务Chubby

- ▶ 2.3.1 Paxos算法
- 2.3.2 Chubby系统设计
- 2.3.3 Chubby中的Paxos
- 2.3.4 Chubby文件系统
- 2.3.5 通信协议
- 2.3.6 正确性与性能

2.3.1 Paxos算法

Paxos算法是Leslie Lamport最先提出的一种基于消息传递(Messages Passing)的一致性算法，用于解决分布式系统中的一致性问题。

在目前所有的一致性算法中，该算法最常用且被认为是最有效的。

2.3 分布式锁服务Chubby

- Paxos算法

简单地说，分布式系统的一致性问题，就是如何保证系统中初始状态相同的各个节点在执行相同的操作序列时，看到的指令序列是完全一致的，并且最终得到完全一致的结果。

怎样才能保证在一个操作序列中每个步骤仅有一个值呢？

2.3 分布式锁服务Chubby

● Paxos算法

一个最简单的方案就是在分布式系统中设置一个专门节点，在每次需要进行操作之前，系统的各个部分向它发出请求，告诉该节点接下来系统要做什么。

该节点接受第一个到达的请求内容作为接下来的操作，这样就能够保证系统只有一个唯一的操作序列。但是这样做也有一个很明显的缺陷，那就是一旦这个专门节点失效，整个系统就很可能出现不一致。

为了避免这种情况，在系统中必然要设置多个专门节点，由这些节点来共同决定操作序列。针对这种多节点决定操作系列的情况，Lamport提出了 Paxos算法。

2.3 分布式锁服务Chubby

• Paxos算法

三个
节点

proposers

提出决议

acceptors

批准决议

learners

获取并使用已经通过的决议

三个
条件

决议只有在被proposers提出后才能批准

每次只批准一个决议

只有决议确定被批准后learners才能获取这个决议

2.3 分布式锁服务Chubby

- Paxos算法

为了满足上述三个条件(主要是第二个条件), 必须对系统有一些约束条件。

Lamport通过约束条件的不断加强, 最后得到了一个可以实际运用到算法中的完整约束条件。

那么, 如何得到这个完整的约束条件呢?

2.3 分布式锁服务Chubby

• Paxos算法

在决议的过程中，proposers将决议发送给acceptors，acceptors对决议进行批准，批准后的决议才能成为正式的决议。决议的批准采用少数服从多数原则，即大多数acceptors接受的决议将成为最终的正式决议。

从集合论的观点来看，两组“多数派”(Majority)至少有一个公共的acceptor。如果每个acceptor只能接受一个决议，则第二个条件就能够得到保证，因此不难得到第一个约束条件--（见下页）

2.3 分布式锁服务Chubby

● 系统的约束条件

p1: 每个acceptor只接受它得到的第一个决议。

p1表明一个acceptor可以收到多个决议，为了区分，对每个决议进行编号，后到的决议编号大于先到的决议编号。

约束条件p1不是很完备，假设系统中一半的acceptors接受了决议1，剩下的一半接受了决议2。此时仅靠约束p1是根本无法得到一个“多数派”，从而无法得到一个正式的决议。进一步加强约束得到：

2.3 分布式锁服务Chubby

- 系统的约束条件

p1: 每个acceptor只接受它得到的第一个决议。

p2: 一旦某个决议得到通过，之后通过的决议必须和该决议保持一致。

p1和p2能够保证第二个条件。对p2稍作加强得到:

p2a: 一旦某个决议v得到通过，之后任何acceptor再批准的决议必须是v。

2.3 分布式锁服务Chubby

● 系统的约束条件

p1: 每个acceptor只接受它得到的第一个决议。

p2: 一旦某个决议得到通过，之后通过的决议必须和该决议保持一致。

p2a: 一旦某个决议 v 得到通过，之后任何acceptor再批准的决议必须是 v 。

表面上看起来已经不存在什么问题了，但实际上p2a和p1是有矛盾的。

考虑下面这种情况:假设在系统得到决议 v 的过程中一个proposer和一个acceptor因为出现问题并没有参与到决议的表决中。在得到决议 v 之后出现问题proposer和acceptor恢复过来，此时这个proposer提出一个决议 w ($w \neq v$)给这个acceptor。如果按照p1，这个acceptor应该接受这个决议 w ，但是按照p2a，则不应该接受这个决议。所以还需进一步加强约束条件。

2.3 分布式锁服务Chubby

- 系统的约束条件

p2: 一旦某个决议得到通过, 之后通过的决议必须和该决议保持一致。

p2a: 一旦某个决议 v 得到通过, 之后任何acceptor再批准的决议必须是 v 。

p2b: 一旦某个决议 v 得到通过, 之后任何proposer再提出的决议必须是 v 。

2.3 分布式锁服务Chubby

● 系统的约束条件

p1: 每个acceptor只接受它得到的第一个决议。

p2: 一旦某个决议得到通过，之后通过的决议必须和该决议保持一致。

p2a: 一旦某个决议v得到通过，之后任何acceptor再批准的决议必须是v。

p2b: 一旦某个决议v得到通过，之后任何proposer再提出的决议必须是v。

满足p1和p2b就能够保证第二个条件，而且彼此之间不存在矛盾。

但是p2b很难通过一种技术手段来实现它，因此提出了一个蕴含p2b的约束p2c.

2.3 分布式锁服务Chubby

● 系统的约束条件

p1: 每个acceptor只接受它得到的第一个决议。

p2: 一旦某个决议得到通过，之后通过的决议必须和该决议保持一致。

p2a: 一旦某个决议 v 得到通过，之后任何acceptor再批准的决议必须是 v 。

p2b: 一旦某个决议 v 得到通过，之后任何proposer再提出的决议必须是 v 。

p2c: 如果一个编号为 n 的提案具有值 v ，那么存在一个“多数派”，要么它们中没有谁批准过编号小于 n 的任何提案，要么它们进行的最近一次批准具有值 v 。

为了保证决议的唯一性，acceptors也要满足一个约束条件：当且仅当acceptors 没有收到编号大于 n 的请求时，acceptors 才批准编号为 n 的提案。

2.3 分布式锁服务Chubby

- 一个决议分为两个阶段

1

准备阶段

proposers选择一个提案并将它的编号设为 n

将它发送给acceptors中的一个“多数派”

acceptors 收到后，如果提案的编号大于它已经回复的所有消息，则acceptors将自己上次的批准回复给proposers，并不再批准小于 n 的提案。

2

批准阶段

当proposers接收到acceptors 中的这个“多数派”的回复后，就向回复请求的acceptors发送accept请求，在符合acceptors一方的约束条件下，acceptors收到accept请求后即批准这个请求。

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

► 2.3.2 Chubby系统设计

2.3.3 Chubby中的Paxos

2.3.4 Chubby文件系统

2.3.5 通信协议

2.3.6 正确性与性能

2.3.2 Chubby系统设计

通常情况下Google的一个数据中心仅运行一个Chubby单元(Chubby cell)，这个单元需要支持包括GFS Bigtable在内的众多Google服务，因此，在设计Chubby时候，必须充分考虑系统需要实现的目标以及可能出现的各种问题。

2.3 分布式锁服务Chubby

- Chubby的设计目标主要有以下几点

1

高可用性和高可靠性

2

高扩展性

3

支持粗粒度的
建议性锁服务

4

服务信息的直接存储

5

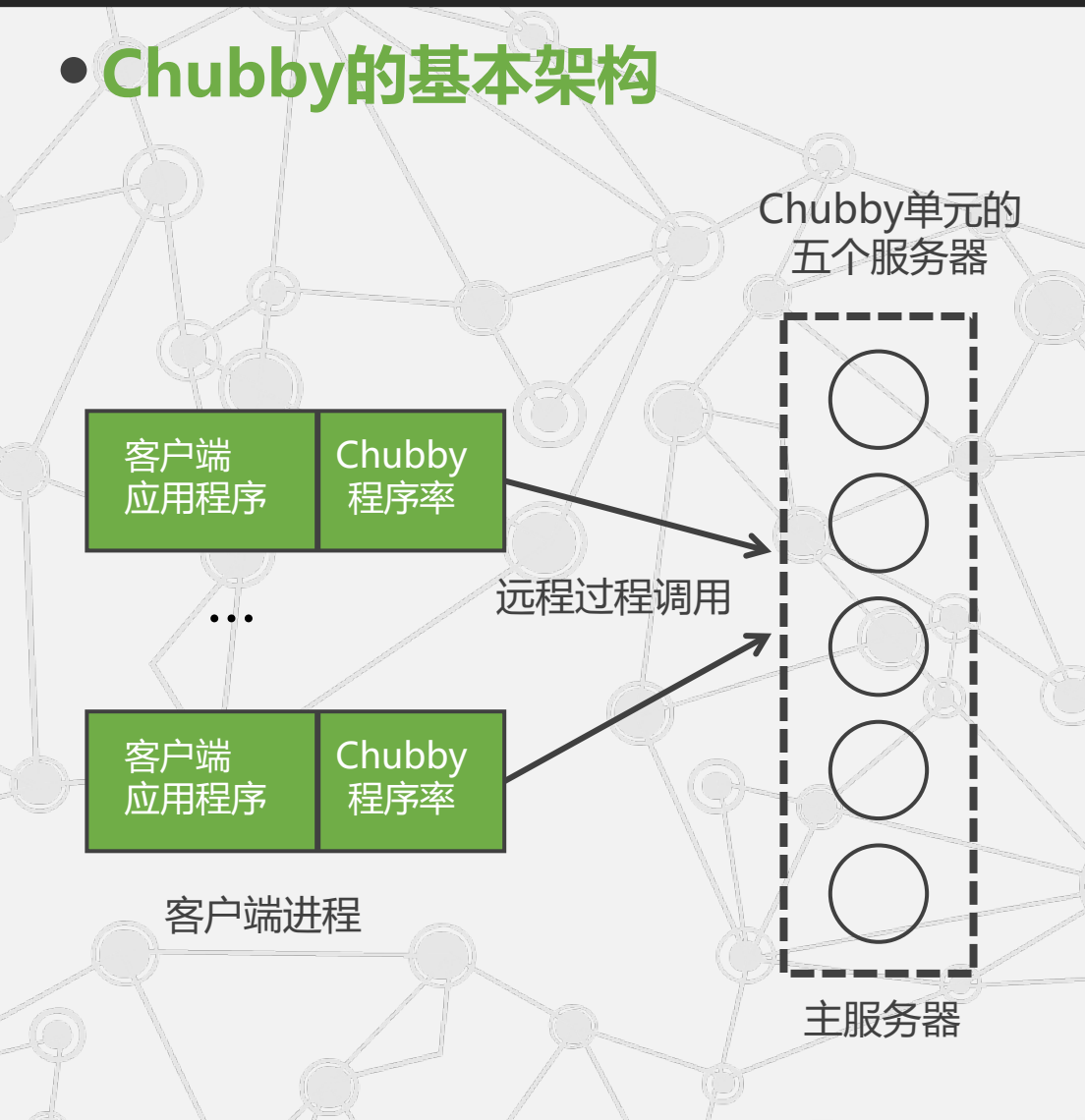
支持缓存机制

6

支持通报机制

2.3 分布式锁服务Chubby

• Chubby的基本架构



客户端

在客户这一端每个客户应用程序都有一个Chubby程序库 (Chubby Library)，客户端的所有应用都是通过调用这个库中的相关函数来完成的。

服务器端

服务器一端称为Chubby单元，一般是由五个称为副本 (Replica) 的服务器组成的，这五个副本在配置上完全一致，并且在系统刚开始时处于对等地位。

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

2.3.2 Chubby系统设计

► 2.3.3 Chubby中的Paxos

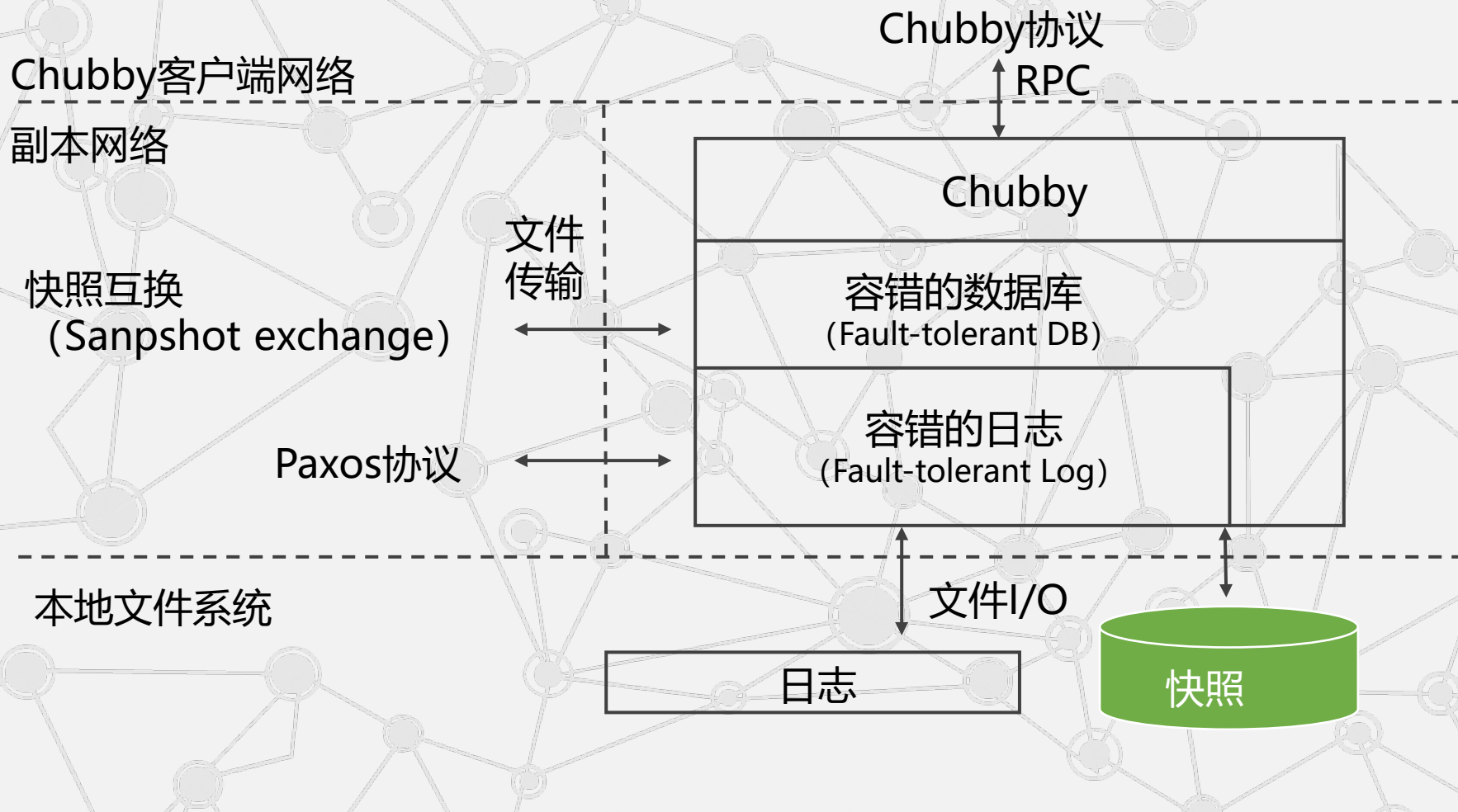
2.3.4 Chubby文件系统

2.3.5 通信协议

2.3.6 正确性与性能

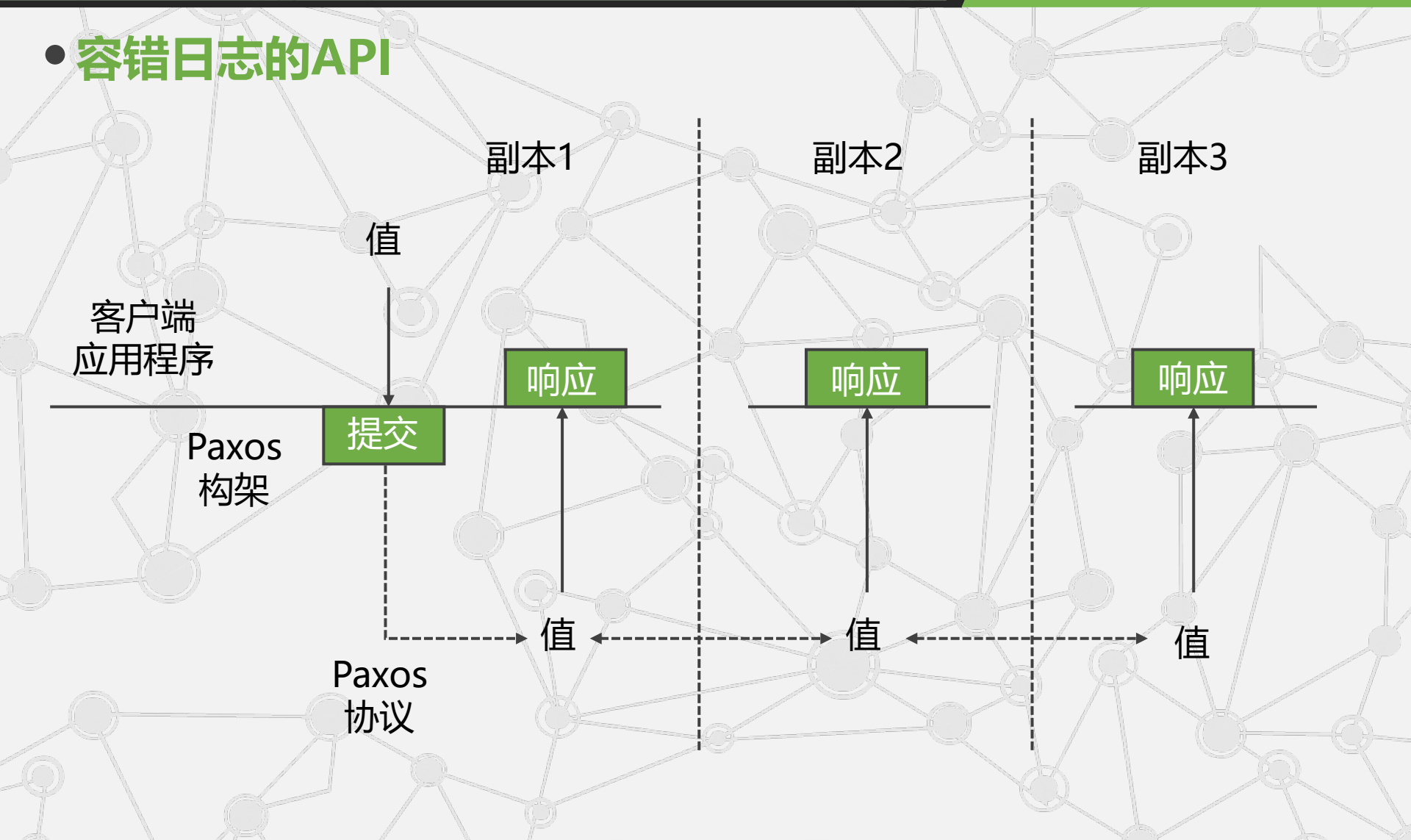
2.3 分布式锁服务Chubby

• 单个Chubby副本结构



2.3 分布式锁服务Chubby

- 容错日志的API



2.3 分布式锁服务Chubby

• 三个过程

(1)选择一个副本成为协调者(Coordinator)。

(2)协调者从客户提交的值中选择一个，然后通过一种被称为accept的消息广播给所有的副本，其他的副本收到广播之后，可以选择接受或者拒绝这个值，并将决定结果反馈给协调者。

(3)一旦协调者收到大多数副本的接受信息后，就认为达到了一致性，接着协调者向相关的副本发送一个commit消息。

上述三个过程实际上跟Paxos的核心思想是完全一致的，这些过程保证提交到不同副本上容错日志中的数据是完全一致的，进而保证Chubby中数据的一致性。

2.3 分布式锁服务Chubby

● 指派序号的方法

(1) 在一个有 n 个副本的系统中，为每个副本分配一个 idi ，其中 $\leq i < n-1$ 。则副本的序号 $s = k \times n + ir$ ，其中 k 的初始值为0。

(2) 某个副本想成为协调者之后，它就根据规则生成一个比它以前的序号更大的序号(实际上就是提高 k 的值)，并将这个序号通过`propose`消息广播给其他所有的副本。

(3) 如果接受到广播的副本发现该序号比它以前见过的序号都大，则向发出广播的副本返回一个`promise`消息，并且承诺不再接受旧的协调者发送的消息。如果大多数副本都返回了`promise`消息，则新的协调者就产生了。

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

2.3.2 Chubby系统设计

2.3.3 Chubby中的Paxos

► 2.3.4 Chubby文件系统

2.3.5 通信协议

2.3.6 正确性与性能

2.3.4 Chubby文件系统

Chubby系统本质上就是一个分布式的、存储大量小文件的文件系统，它所有的操作都是在文件的基础上完成的。

例如在Chubby最常用的锁服务中，每一个文件就代表了一个锁，用户通过打开、关闭和读取文件，获取共享(Shared)锁或独占(Exclusive)锁。选举主服务器的过程中，符合条件的服务器都同时申请打开某个文件并请求锁住该文件。成功获得锁的服务器自动成为主服务器并将其地址写入这个文件夹，以便其他服务器和用户可以获知主服务器的地址信息。

2.3 分布式锁服务Chubby

- 单调递增的64位编号

1 实例号

Instance Number

新节点实例号必定大于旧节点的实例号。

2 内容生成号

Content Generation Number

文件内容修改时该号增加。

3 锁生成号

Lock Generation Number

锁被用户持有时该号增加。

4 ACL生成号

ACL Generation Number

ACL名被覆写时该号增加。

2.3 分布式锁服务Chubby

● 常用的句柄函数及作用

函数名称	作用
Open()	打开某个文件或者目录来创建句柄
Close()	关闭打开的句柄，后续的任何操作都将中止
Poison()	中止当前未完成及后续的操作，但不关闭句柄
GetContentsAndStat()	返回文件内容及元数据
GetStat()	只返回文件元数据
ReadDir()	返回子目录名称及其元数据
SetContents()	向文件中写入内容
SetACL()	设置ACL名称
Delete()	如果该节点没有子节点的话则执行删除操作
Acquire()	获取锁
Release()	释放锁
GetSequencer()	返回一个sequencer
SetSequencer()	将sequencer和某个句柄进行关联
CheckSequencer()	检查某个sequencer是否有效

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

2.3.2 Chubby系统设计

2.3.3 Chubby中的Paxos

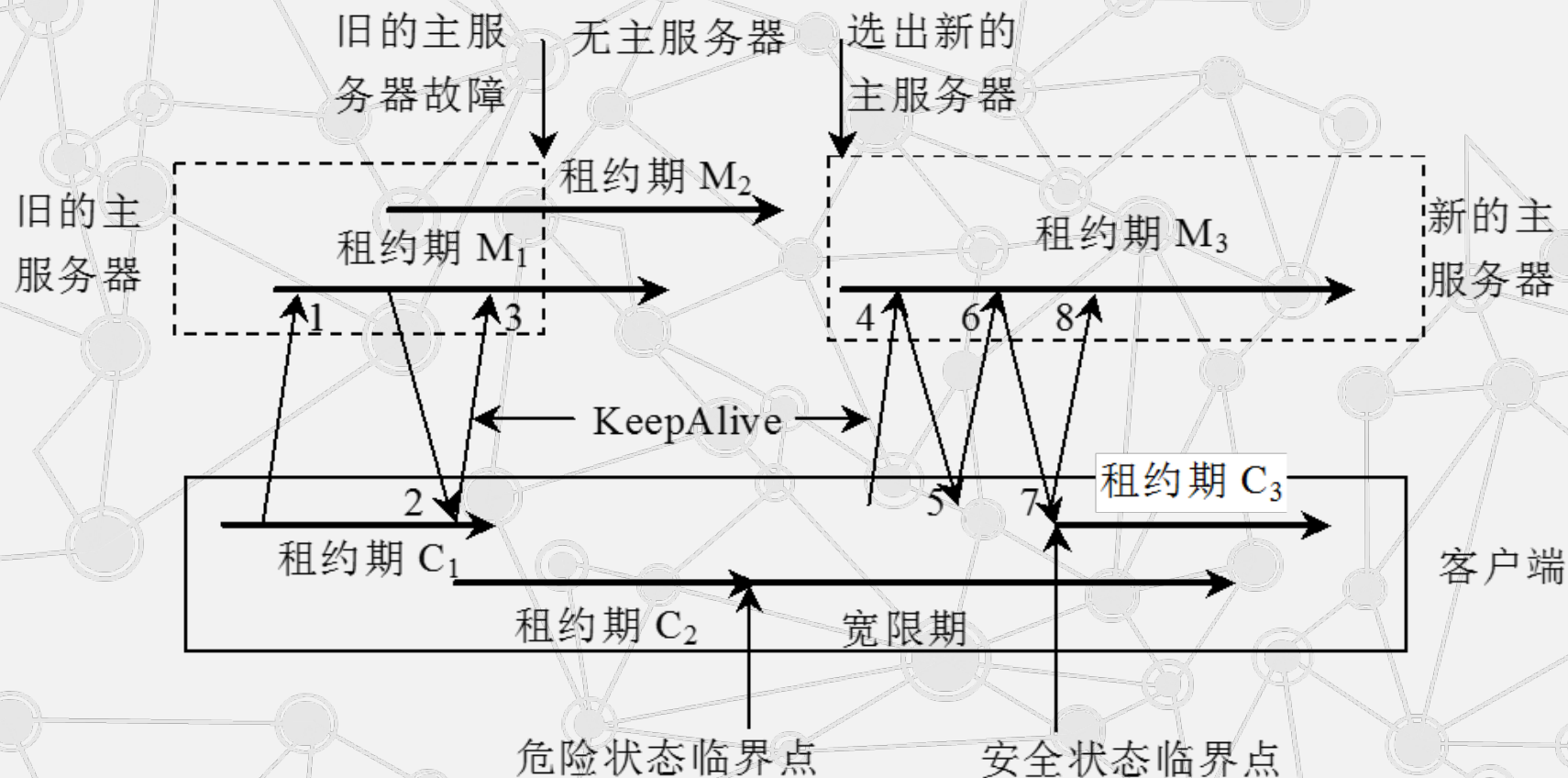
2.3.4 Chubby文件系统

► 2.3.5 通信协议

2.3.6 正确性与性能

2.3 分布式锁服务Chubby

• Chubby客户端与服务器端的通信过程



2.3 分布式锁服务Chubby

- 可能出现两种故障



1

客户端租约过期



2

主服务器出错

2.3 分布式锁服务Chubby

2.3.1 Paxos算法

2.3.2 Chubby系统设计

2.3.3 Chubby中的Paxos

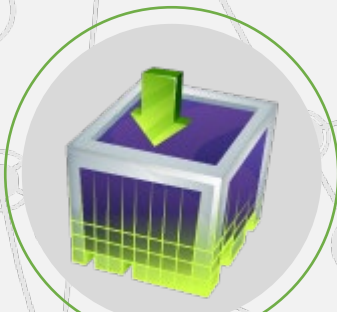
2.3.4 Chubby文件系统

2.3.5 通信协议

▶ 2.3.6 正确性与性能

2.3 分布式锁服务Chubby

• 正确性与性能



一致性

每个Chubby单元是由五个副本组成的，这五个副本中需要选举产生一个主服务器，这种选举本质上就是一个一致性问题



安全性

采用的是ACL形式的安全保障措施。只要不被覆盖，子节点都是直接继承父节点的ACL名

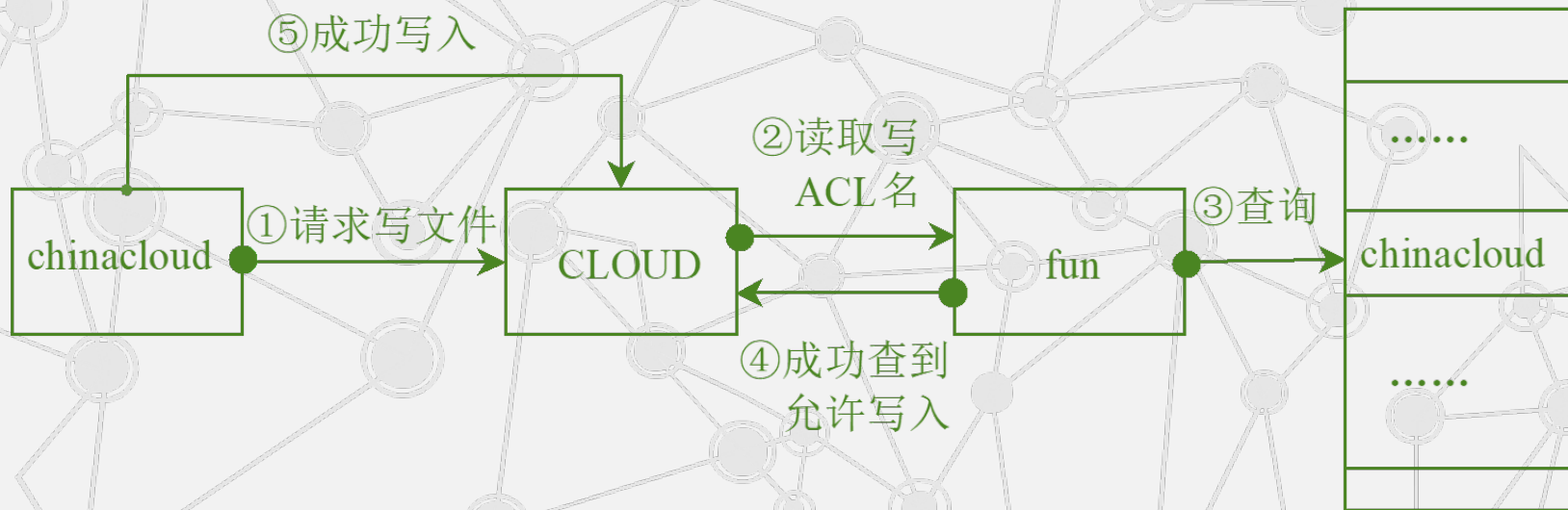


性能优化

提高主服务器默认的租约期、使用协议转换服务将Chubby协议转换成较简单的协议、客户端一致性缓存等

2.3 分布式锁服务Chubby

• Chubby 的 ACL 机制



用户chinacloud提出向文件CLOUD中写入内容的请求。CLOUD首先读取自身的写ACL名fun，接着在fun中查到了chinacloud这一行记录，于是返回信息允许chinacloud对文件进行写操作，此时chinacloud才被允许向CLOUD写入内容。其他的操作和写操作类似。

目录

2.1 Google文件系统GFS

2.2 分布式数据处理MapReduce

2.3 分布式锁服务Chubby

2.4 分布式结构化数据表Bigtable

2.5 分布式存储系统Megastore

2.6 大规模分布式系统的监控基础架构Dapper

2.7 海量数据的交互式分析工具Dremel

2.8 内存大数据分析系统PowerDrill

2.9 Google应用程序引擎

2.4 分布式结构化数据表Bigtable

Bigtable是Google开发的基于 GFS和Chubby的分布式存储系统。Google的很多数据，包括Web 索引、卫星图像数据等在内的海量结构化和半结构化数据，都存储在 Bigtable中。从实现上看，Bigtable并没有什么全新的技术，但是如何选择合适的技术并将这些技术高效、巧妙地结合在一起恰恰是最大的难点。Bigtable在很多方面和数据库类似，但它并不是真正意义上的数据居库。

通过本节的学习，对Bigtable的数据模型、系统架构、实现以及它使用的的一些数据库技术有一个全面的认识。

2.4 分布式结构化数据表Bigtable

- ▶ 2.4.1 设计动机与目标
- 2.4.2 数据模型
- 2.4.3 系统架构
- 2.4.4 主服务器
- 2.4.5 子表服务器
- 2.4.6 性能优化

2.4 分布式结构化数据表Bigtable

• Bigtable 的设计动机

Google运行着目前世界上最繁忙的系统，它每时每刻处理的客户服务请求数量是普通的系统根本无法承受的

包括URL、网页内容、用户的个性化设置在内的数据都是Google需要经常处理的

需要存储
的数据种类繁多

2

海量的
服务请求

3

一方面现有商用数据库的设计着眼点在于其通用性。另一方面对于底层系统的完全掌控会给后期的系统维护、升级带来极大的便利

商用数据库
无法满足需求

2.4 分布式结构化数据表Bigtable

- **Bigtable 应达到的基本目标**

广泛的适用性

Bigtable是为了满足一系列Google产品而并非特定产品的存储要求。

很强的可扩展性

根据需要随时可以加入或撤销服务器

高可用性

确保几乎所有的情况下系统都可用

简单性

底层系统的简单性既可以减少系统出错的概率，也为上层应用的开发带来便利

2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

► 2.4.2 数据模型

2.4.3 系统架构

2.4.4 主服务器

2.4.5 子表服务器

2.4.6 性能优化

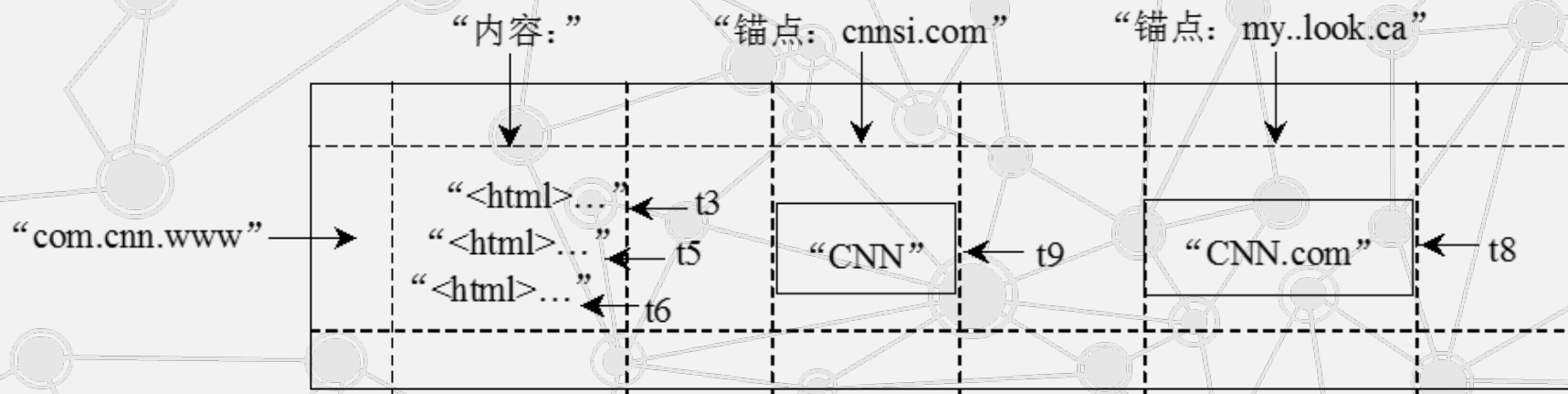
2.4 分布式结构化数据表Bigtable

- Bigtable数据的存储格式

Bigtable是一个分布式多维映射表，表中的数据通过一个行关键字（Row Key）、一个列关键字（Column Key）以及一个时间戳（Time Stamp）进行索引

Bigtable的存储逻辑可以表示为：

(row:string, column:string, time:int64)→string



2.4 分布式结构化数据表Bigtable

行

- Bigtable的行关键字可以是任意的字符串，但是大小不能够超过64KB
- 表中数据都是根据行关键字进行排序的，排序使用的是词典序
- 同一地址域的网页会被存储在表中的连续位置
- 倒排便于数据压缩，可以大幅提高压缩率

列

- 将其组织成所谓的列族 (Column Family)
- 族名必须有意义，限定词则可以任意选定
- 组织的数据结构清晰明了，含义也很清楚
- 族同时也是Bigtable中访问控制 (Access Control) 的基本单元

时间戳

- Google的很多服务比如网页检索和用户的个性化设置等都需要保存不同时间的数据，这些不同的数据版本必须通过时间戳来区分。
- Bigtable中的时间戳是64位整数，具体的赋值方式可以由用户自行定义

2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

2.4.2 数据模型

▶ 2.4.3 系统架构

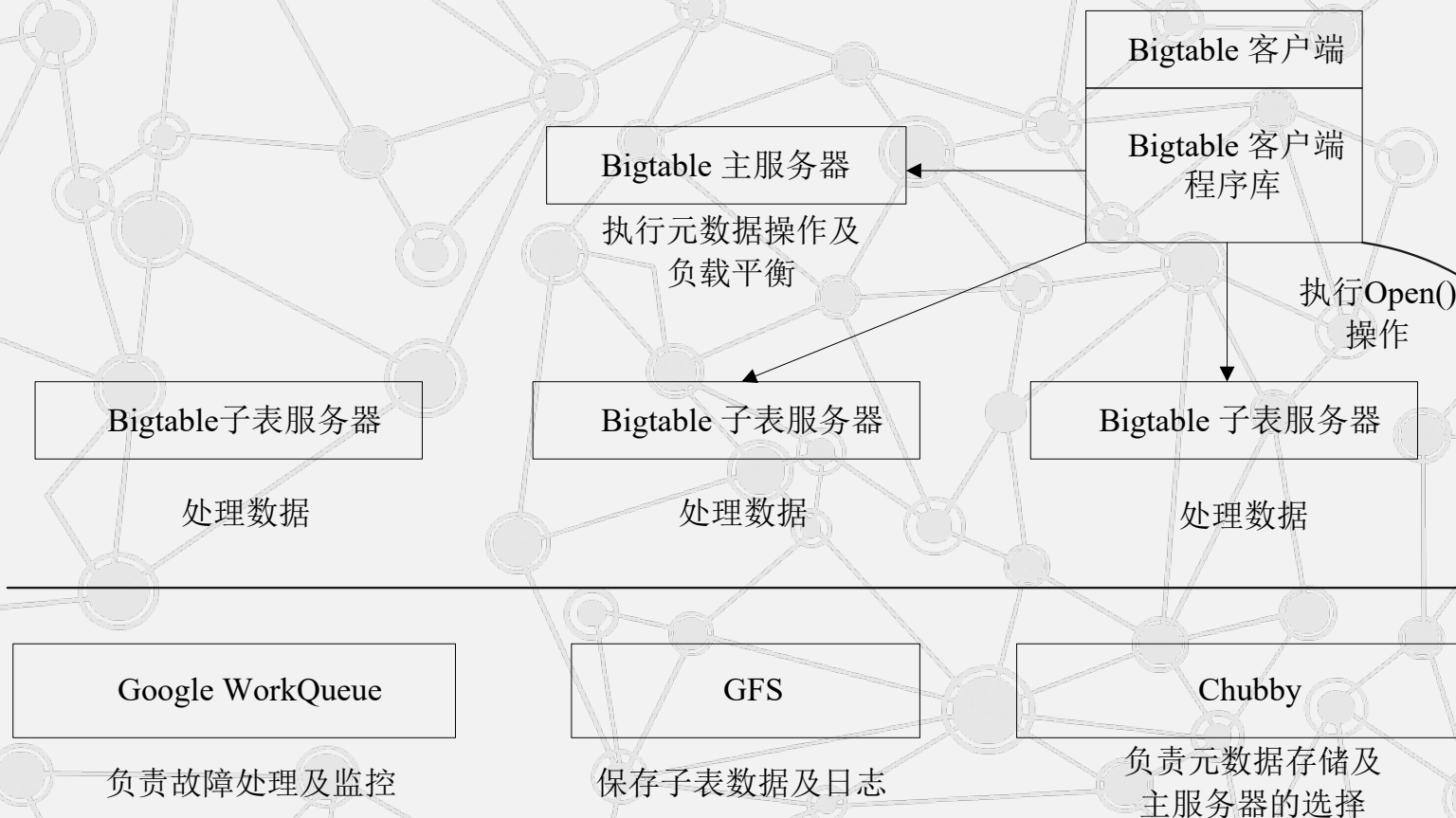
2.4.4 主服务器

2.4.5 子表服务器

2.4.6 性能优化

2.4 分布式结构化数据表Bigtable

• Bigtable 基本架构



2.4 分布式结构化数据表Bigtable

- Bigtable 中 Chubby 的主要作用

作用一

选取并保证同一时间内只有一个主服务器 (Master Server) 。

作用二

获取子表的位置信息。

作用三

保存Bigtable的模式信息及访问控制列表。

2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

2.4.2 数据模型

2.4.3 系统架构

► 2.4.4 主服务器

2.4.5 子表服务器

2.4.6 性能优化

2.4 分布式结构化数据表Bigtable



2.4 分布式结构化数据表Bigtable

• Bigtable 中 Chubby 的主要作用

从Chubby中获取一个独占锁，确保同一时间只有一个主服务器

步骤 1

步骤 2

扫描服务器目录，发现目前活跃的子表服务器

与所有的活跃子表服务器取得联系以便了解所有子表的分配情况

步骤 3

步骤 4

通过扫描元数据表 (Metadata Table)，发现未分配的子表并将其分配到合适的子表服务器

2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

2.4.2 数据模型

2.4.3 系统架构

2.4.4 主服务器

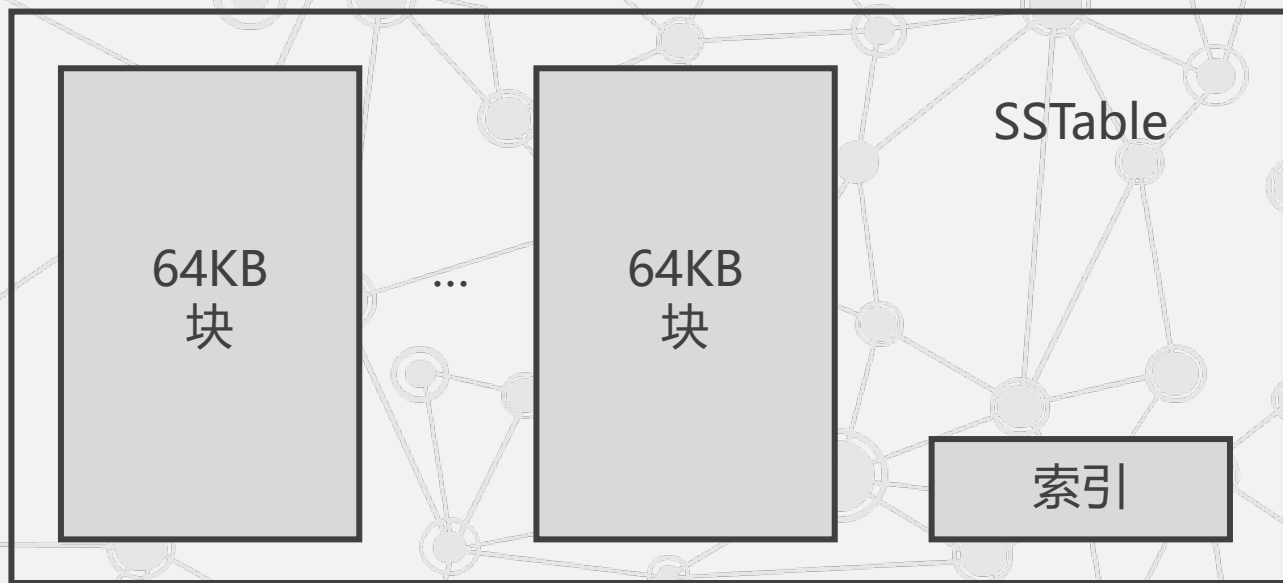
► 2.4.5 子表服务器

2.4.6 性能优化

2.4 分布式结构化数据表Bigtable

- **SSTable 格式的基本示意**

SSTable是Google为Bigtable设计的内部数据存储格式。所有的SSTable文件都存储在GFS上，用户可以通过键来查询相应的值。



2.4 分布式结构化数据表Bigtable

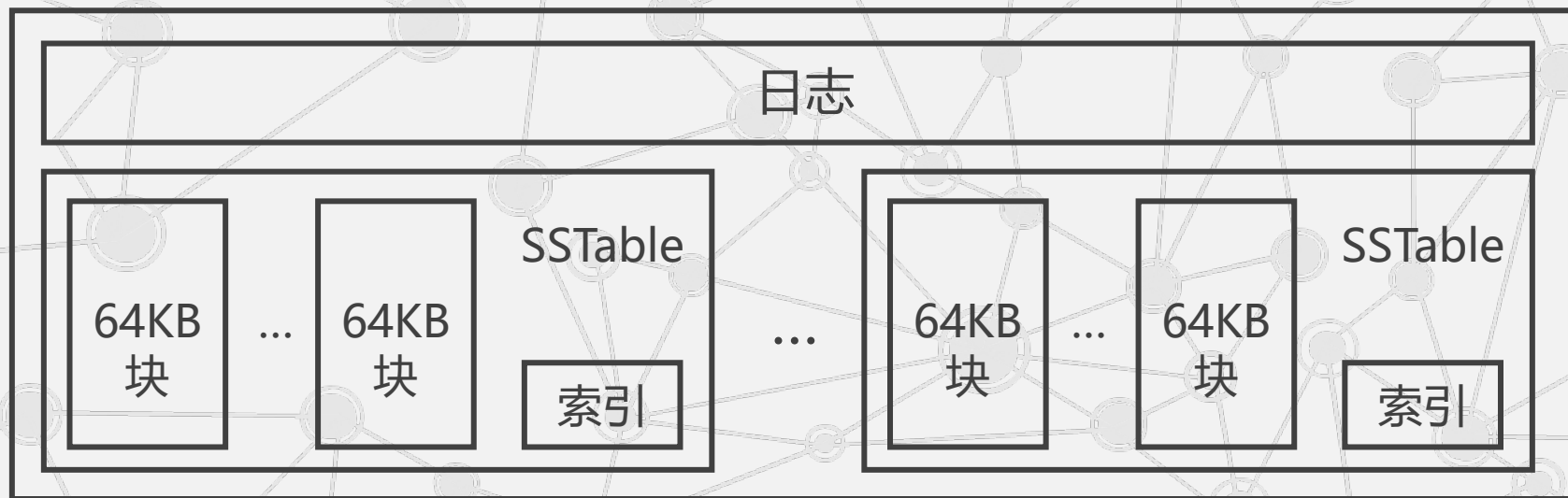
● 子表实际组成

不同子表的SSTable可以共享

每个子表服务器上仅保存一个日志文件

Bigtable规定将日志的内容按照键值进行排序

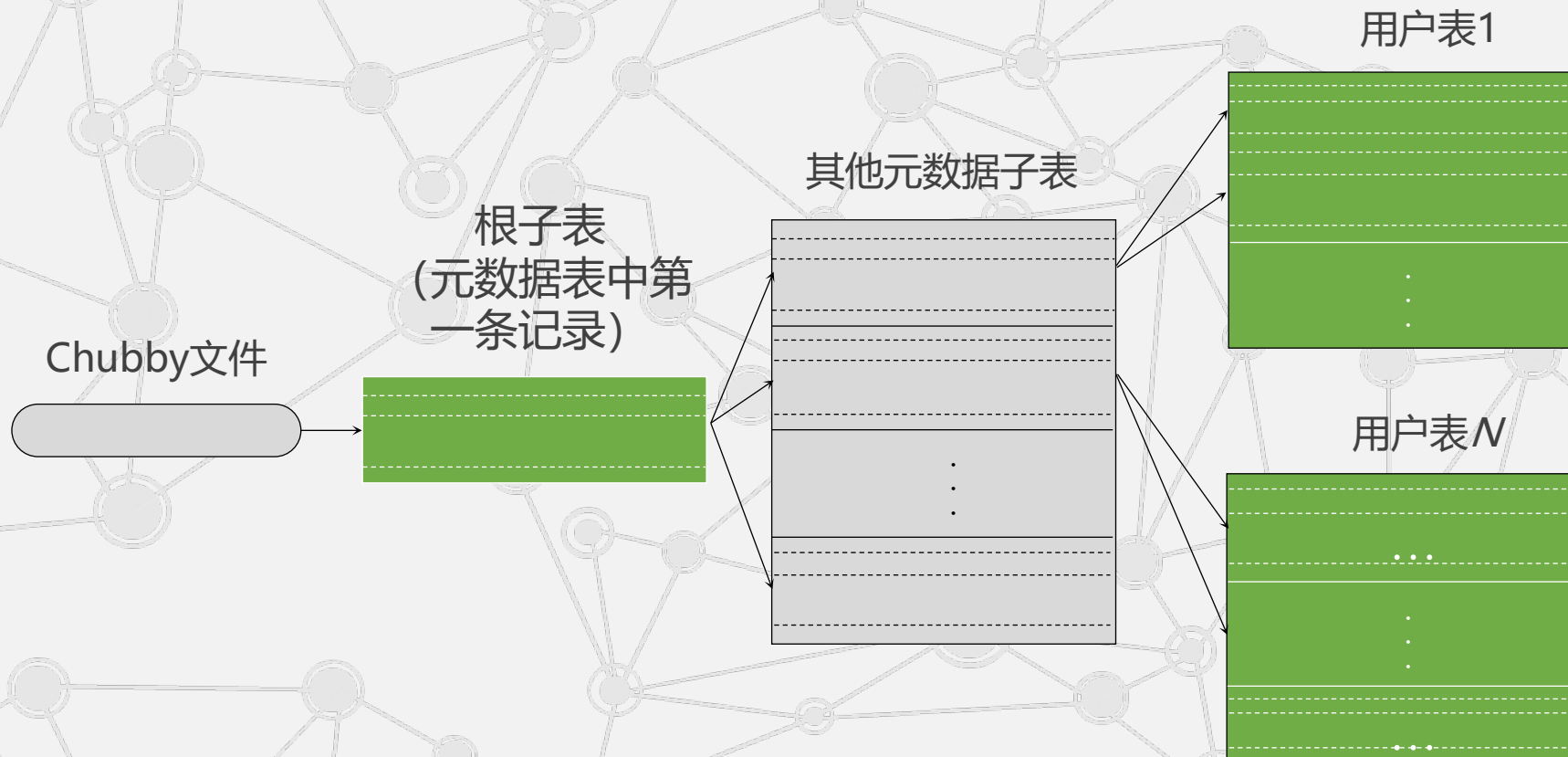
每个子表服务器上保存的子表数量可以从几十到上千不等，通常情况下是100个左右



2.4 分布式结构化数据表Bigtable

● 子表地址组成

Bigtable系统的内部采用的是一种类似B+树的三层查询体系

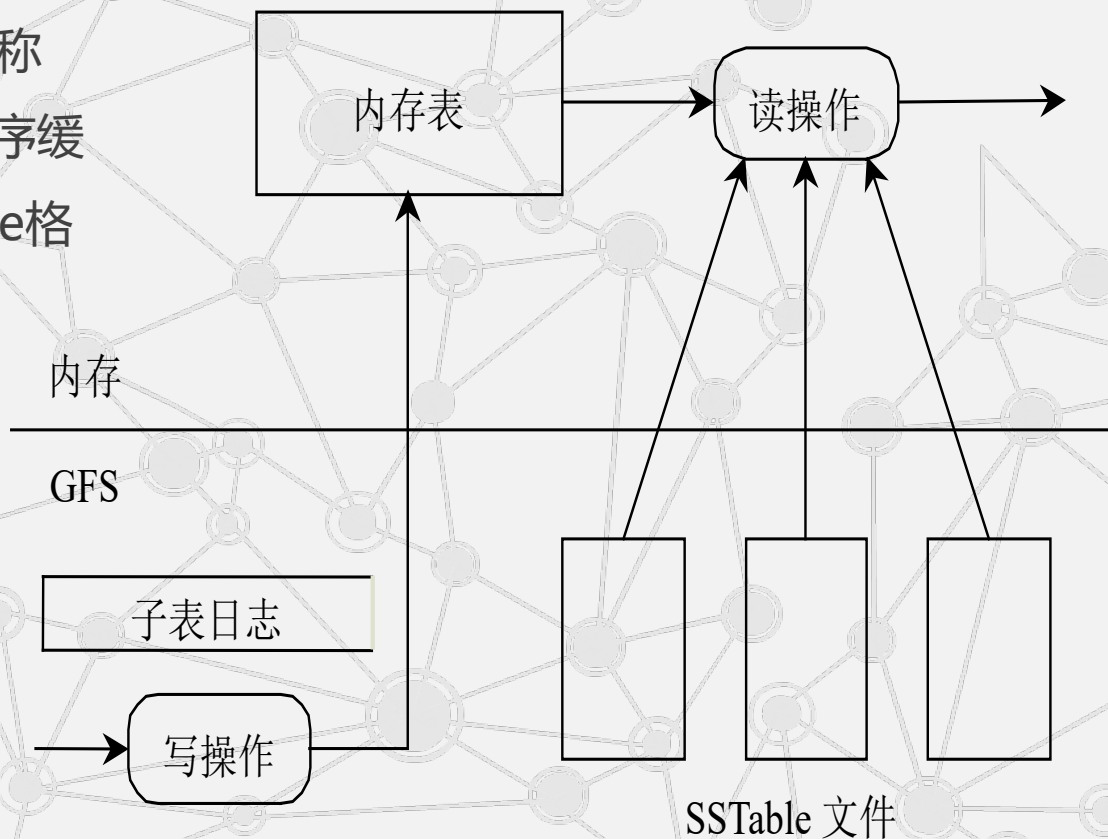


2.4 分布式结构化数据表Bigtable

• Bigtable 数据存储及读/写操作

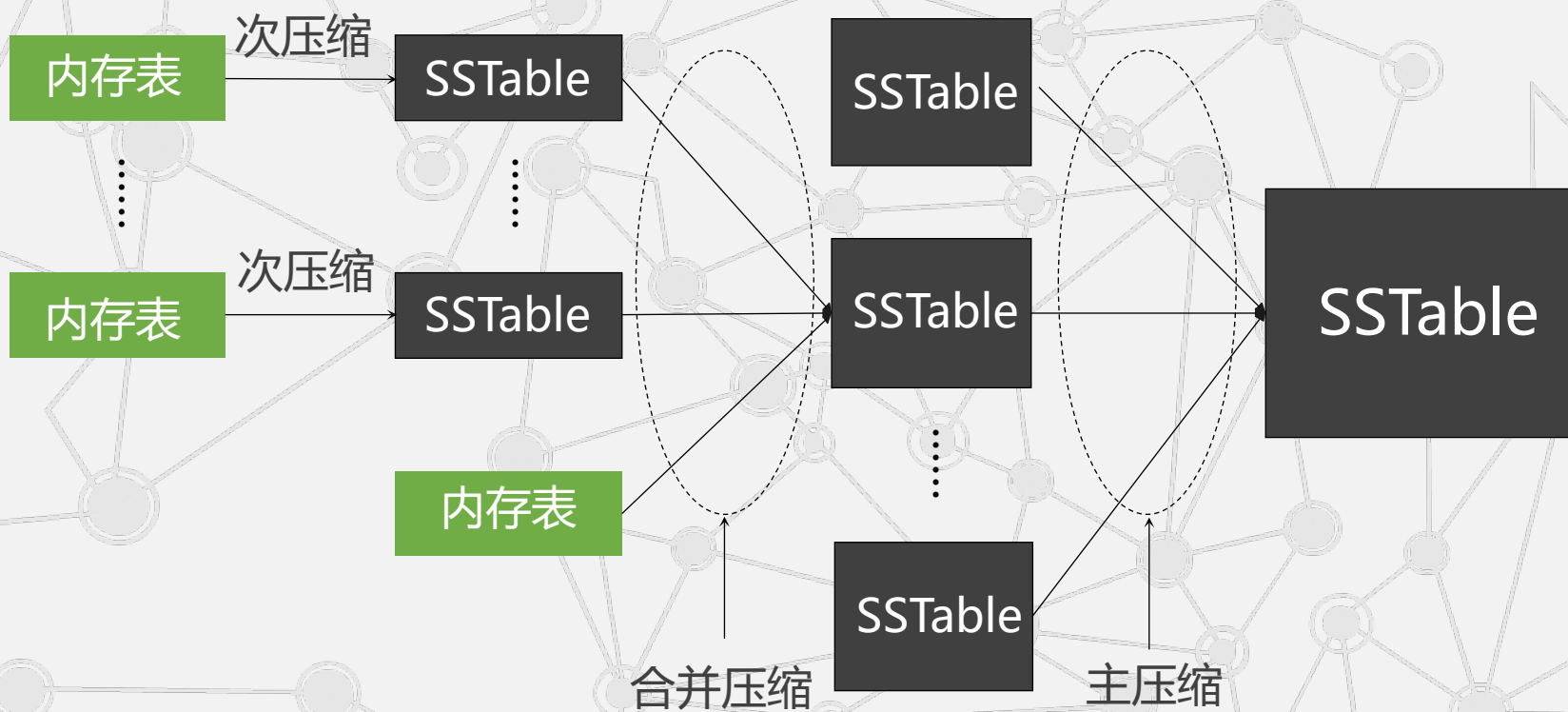
较新的数据存储在内存中一个称为内存表 (Memtable) 的有序缓冲里，较早的数据则以SSTable格式保存在GFS中。

读和写操作有很大的差异性



2.4 分布式结构化数据表Bigtable

- 三种形式压缩之间的关系



2.4 分布式结构化数据表Bigtable

2.4.1 设计动机与目标

2.4.2 数据模型

2.4.3 系统架构

2.4.4 主服务器

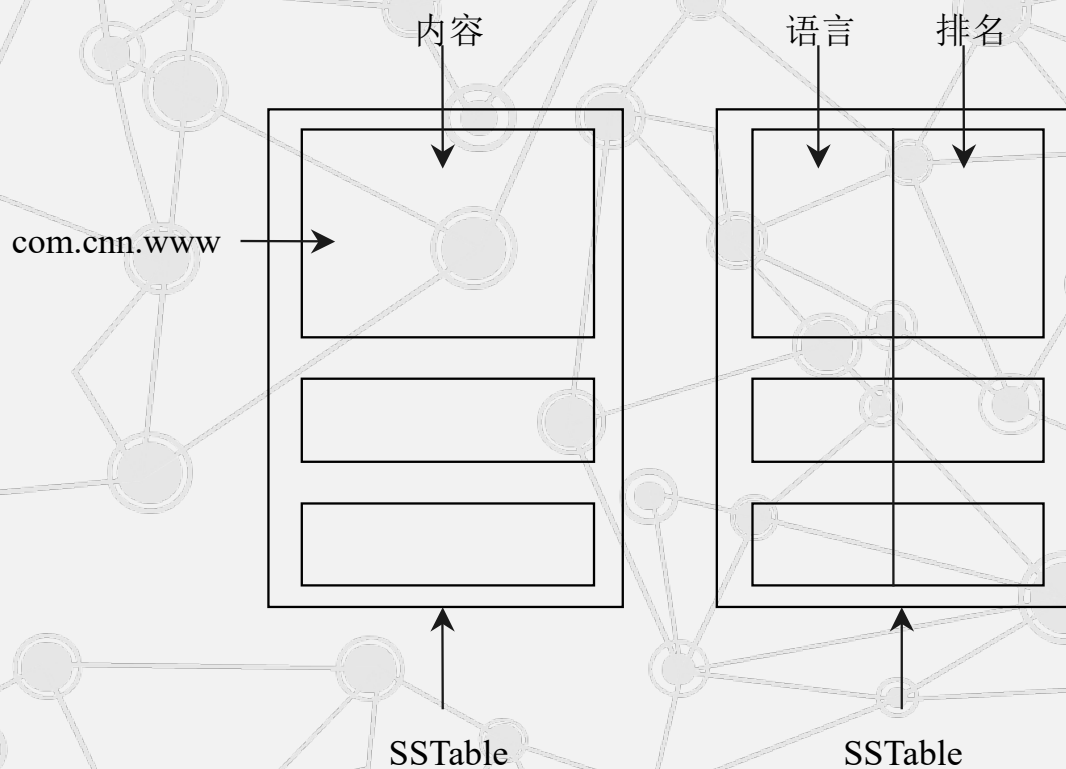
2.4.5 子表服务器

► 2.4.6 性能优化

2.4 分布式结构化数据表Bigtable

● 局部性群组

Bigtable允许用户将原本并不存储在一起的数据以列族为单位，根据需要组织在一个单独的SSTable中，以构成一个局部性群组。



用户可以只看自己感兴趣的内容。

对于一些较小的且会被经常读取的局部性群组，明显地改善读取效率。

2.4 分布式结构化数据表Bigtable

• 压缩

压缩可以有效地节省空间，Bigtable中的压缩被应用于很多场合。首先压缩可以被用在构成局部性群组的SSTable中，可以选择是否对个人的局部性群组的SSTable进行压缩。

1

利用Bentley & McIlroy方式（BMDiff）在大的扫描窗口将常见的长串进行压缩

2

采取Zippy技术进行快速压缩，它在一个16KB大小的扫描窗口内寻找重复数据，这个过程非常快

2.4 分布式结构化数据表Bigtable

● 布隆过滤器

Bigtable向用户提供了一种称为**布隆过滤器**的数学工具。布隆过滤器是巴顿·布隆在**1970**年提出的，实际上它是一个很长的**二进制向量**和一系列**随机映射函数**，在读操作中确定子表的位置时非常有用。

优点

- 布隆过滤器的速度快，省空间
- 不会将一个存在的子表判定为不存在

缺点

- 在某些情况下它会将不存在的子表判断为存在

2.4 分布式结构化数据表Bigtable

• 从Bigtable到向量数据库——数据存储的新维度

- > - Bigtable开创了NoSQL列存储的先河，现代发展：Google Spanner（全球分布式关系数据库）、Apache Cassandra、HBase
- > - LLM时代的新需求：向量数据库（Vector Database）
- > - 大模型通过"嵌入"（Embedding）将文本/图像转为高维向量
- > - 向量数据库专门存储和高效检索这些向量，支撑**RAG（检索增强生成）**应用
- > - 代表系统：Google Vertex AI Vector Search、Pinecone、Milvus、Weaviate
- > - Bigtable的分布式存储思想为向量数据库的横向扩展奠定了基础



本章未完待续



云计算 (第三版)

CLOUD COMPUTING Third Edition

第2章

谢谢观看